

PneumoniaMNIST classification

comparison of logistic regression and convolutional neural networks

J. Stoffin
G. Valcheva

AI programming brush-up project
June 18, 2026

This project implements a small but complete medical machine learning workflow using PneumoniaMNIST from MedMNIST. The aim was not to reach state-of-the-art performance, but to build a reproducible experiment that covers the full path from dataset loading to final test evaluation. We compared a classical scikit-learn logistic regression baseline with two PyTorch convolutional neural networks: a standard small CNN and a class-weighted CNN. The official train, validation, and test splits were used throughout. The validation set was used for model selection, while the test set was reserved for final evaluation only. The best overall result was obtained by the class-weighted CNN, which reached a test AUC of 0.936 and accuracy of 0.881. The results show that the CNN models learned useful image structure, but the improvement over logistic regression was moderate, which is reasonable given the low 28 by 28 image resolution and the small architecture.

1 Problem set up and data handling

The goal of this project was to apply the programming workflow of a small medical AI study, showcasing collaboration, version-controlled contribution, strong ownership and methodological choice reasoning. To achieve it, we implemented a binary classification analysis via classical machine learning baseline in scikit-learn (logistic regression), and compared it with a modular deep learning model in PyTorch (weighted/small Convolutional Neural Network-CNN), focusing on correctness, reproducibility, and understanding. The comparison between logistic regression and a CNN was chosen because the two models use the image information differently. Logistic regression treats the image as a flat vector of pixel values. A CNN keeps the spatial structure of the image and can learn local patterns such as edges, textures, and small intensity changes. The project uses the PneumoniaMNIST dataset, as it is medically meaningful and suitable for the project objective.

Data procurement The dataset was loaded through the MedMNIST Python library. Link: <https://medmnist.com>

Data description The PneumoniaMNIST dataset consists of a sample of 5,856 grayscale chest x-rays resized to 28×28 pixels. Each image belongs to one of two classes: normal or pneumonia, making the dataset suitable for the objective of a supervised binary classification.

The class distribution is moderately imbalanced, with pneumonia cases occurring more frequently than normal cases (Table 2, Appendix E). This motivated the inclusion of a class-weighted CNN variant in addition to the standard CNN. Additionally, due to the imbalance we report not only accuracy, but supporting nuance as presented by the F1-score, AUC, sensitivity, specificity, and confusion matrices.

Data preprocessing Images were represented as single-channel intensity matrices. Pixel values were subsequently normalised before model training. For the logistic regression baseline, images were flattened into vectors of length 784. For the CNN models, the original spatial structure was preserved.

Train/validation/test split We used the official train, validation, and test splits provided by MedMNIST. This is important because it prevents accidental data leakage and makes the experiment easier to reproduce. The training set contained 4,708 images, the validation set contained 524 images, and the test set contained 624 images. The training split was used to fit the models, the validation split was used for hyperparameter optimisation and checkpoint selection, and the test split was used only for final evaluation.

2 Reproducibility and project structure

Repository structure The project repository is organised to separate reusable code (models, training, evaluation, etc.), configuration, experiments, generated outputs, figures, reports, tests, and Snellius scripts. The detailed file structure is available in the Appendix C, while the repository is available at:

<https://github.com/GalinaValcheva/9A2-brushup-group10/tree/main>

Experiment records Generated files such as checkpoints, logs, and raw JSON metrics are written to `outputs/`. This folder is ignored by Git except for a placeholder file, because these outputs can be regenerated and may become large. The final comparison table is stored in `experiments/`, and report figures are stored in `figures/`.

Documentation The repository is documented through a detailed README (as well as, again – configuration files, experiment summaries) and a final report that together describe the dataset, implementation, evaluation protocol, and instructions required to reproduce the main results.

Version control Version control was managed using Git and GitHub, allowing incremental development, transparent tracking of code changes, and collaborative work while maintaining a reproducible project history.

Reproducibility Reproducibility was supported through various methods. For example, the use of fixed random seeds and the official MedMNIST train, validation, and test splits, allowing experiments to be rerun under consistent conditions while avoiding data leakage. To reproduce the project locally follow Appendix B: Exact run commands.

3 Baseline model

The baseline model was logistic regression implemented in scikit-learn. It was combined with standardisation, because logistic regression is sensitive to feature scaling. An evaluation identical to the deep learning implementation evaluation has been performed and outlined later.

Tuning The regularisation strength C was tuned using the validation set only. The tested values were:

0.01, 0.1, 1.0, 10.0

Logistic regression was selected as the baseline because it represents an elegant first-principles approach to classification: it is mathematically transparent, statistically well understood, and establishes a strong reference point before introducing additional model complexity. Many modern classification methods can be viewed as extensions, relaxations, or nonlinear generalisations of the same underlying probabilistic framework, making logistic regression a particularly informative baseline. By starting with a model whose behaviour, assumptions, and decision boundaries can be explained directly, any performance gains achieved by the CNN can be attributed more confidently to the exploitation of spatial image structure rather than simply to an increase in model complexity.

4 Deep learning implementation

The deep learning model was a small CNN implemented in PyTorch. The architecture contains two convolutional blocks followed by a small fully connected classifier. It is intentionally modest because the project emphasises correctness and understanding rather than state-of-the-art performance. Two approaches – regular and weighted small CNN were implemented.

The CNN receives a $1 \times 28 \times 28$ grayscale image. The first convolutional block learns 16 feature maps, followed by ReLU and max pooling. The second block learns 32 feature maps, again followed by ReLU and max pooling. After two pooling operations, the spatial size is reduced to 7×7 , and the classifier maps the learned representation to two output logits.

4.1 Small CNN

Loss The loss function was cross entropy loss.

Optimizer The optimiser was Adam with a learning rate of 0.001.

Training loop The model was trained for 10 epochs.

Validation procedure After every epoch, validation metrics were computed.

Checkpoints The best checkpoint was saved based on the highest validation AUC, not the final epoch, as the last one is not necessarily the epoch where the model generalises best.

4.2 Small weighted CNN

A class-weighted CNN was also trained. It used the same architecture, but the cross entropy loss included class weights computed from the training labels. This was tested because the dataset is imbalanced, and class weighting can reduce the tendency of the model to focus too much on the majority class.

5 Evaluation and interpretation

This section reports relevant results, compares the models, and discusses performance, errors, limitations, and possible improvements.

Evaluation All models were evaluated using the same final test split. The reported metrics for all models were accuracy, F1-score, AUC, sensitivity, specificity, and confusion matrix.

Accuracy gives the proportion of correct predictions. F1-score balances precision and recall. AUC measures how well the model separates the two classes across thresholds, which is useful when the positive decision threshold may change. Sensitivity is especially relevant medically because it measures how many pneumonia cases are detected. Specificity measures how many normal cases are correctly recognised.

5.1 Results

Table 1: Comparison: test set performance of the three models

model	accuracy	f1	auc	sensitivity	specificity
logistic regression	0.8734	0.9056	0.9298	0.9718	0.7094
small CNN	0.8718	0.9061	0.9337	0.9897	0.6752
small CNN class-weighted	0.8814	0.9121	0.9359	0.9846	0.7094

The class-weighted CNN achieved the highest accuracy, F1-score, and AUC. The standard CNN had slightly higher AUC than logistic regression, but slightly lower accuracy. This shows that the CNN learned useful ranking/separation information, but the improvement was not dramatic. The weighted CNN gave the best overall balance, suggesting that class imbalance mattered for this task.

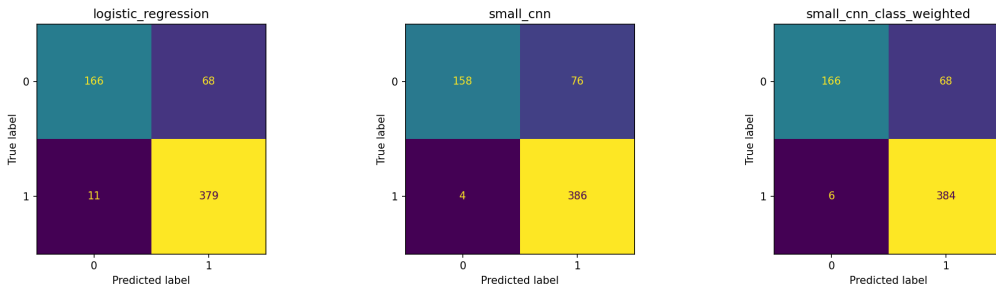


Figure 1: Confusion matrices for logistic regression, the standard CNN, and the class-weighted CNN on the held-out test set.

As portrayed in Appendix D, Figure 3, the logistic regression baseline was tuned on the validation set by varying the regularisation parameter (C). The highest validation AUC was achieved at ($C=0.01$) (AUC = 0.9916), while performance declined gradually to 0.9840 at ($C=10$), indicating that stronger regularisation improved generalisation. Validation log loss showed a similar pattern, reaching its minimum at ($C=0.1$) (0.121) before increasing to 0.340 at ($C=10$), suggesting that although class separation remained strong, probability estimates became substantially less reliable as regularisation weakened. The same figure contains training curves of both CNN models, showcasing they learned quickly. Validation AUC was already high after a 2 to 4 epochs, and the later gains were small. This suggests that the task is learnable with a small architecture, but also that more epochs alone would probably not be the most meaningful improvement. The confusion matrices in Figure 1 show that all models detect most pneumonia cases, but there are still errors in the normal class. This is reflected by the lower specificity compared with sensitivity.

5.2 Interpretation and discussion

The weighted CNN performed best overall, with test AUC of 0.9359 and accuracy of 0.8814. This was expected to some extent because the CNN can use spatial information and the class-weighted loss helps address imbalance. However, the improvement over logistic regression was moderate rather than substantial. It shows that for this low-resolution dataset, even a simple linear baseline can extract useful signal from flattened pixel intensities.

The strongest medical concern is the trade-off between sensitivity and specificity. The CNN models had very high sensitivity, meaning they detected most pneumonia cases. This is clinically valuable because missing pneumonia can be harmful. However, specificity was lower, which means some normal cases were classified as pneumonia. In a screening context this might be acceptable, but in a diagnostic setting it would need further validation and probably threshold calibration.

The weighted CNN improved the overall balance. It kept sensitivity high while improving specificity compared with the standard CNN. This suggests that class weighting helped the model pay more attention to the class distribution rather than simply optimising for the dominant pattern, ultimately resulting in the best model performance.

Limitations There are several limitations. First, the images are only 28 by 28 pixels, so clinically relevant detail is lost. Second, the CNN is deliberately small and was not heavily tuned. Third, there was no external validation on another dataset, so the results only describe performance on the MedMNIST test split. Fourth, the model outputs were not calibrated, so predicted probabilities should not be interpreted as reliable clinical risk estimates. Finally, this project is a programming and evaluation exercise, not a clinical deployment study.

Possible improvements Future work could include stronger data augmentation, a deeper CNN, transfer learning from a model pretrained on larger medical or natural image datasets, threshold tuning based on clinical priorities, probability calibration, and external validation. It would also be useful to save richer experiment metadata, such as Git commit hash, runtime, hardware, and exact package versions for every run. Another useful extension would be error analysis by inspecting misclassified images, especially false negatives and false positives.

Conclusion This project demonstrates a complete small medical AI workflow: loading a MedMNIST dataset, protecting train-validation-test splits, preprocessing data, training a classical baseline, training PyTorch CNN models, saving checkpoints, evaluating on a final test set, and documenting the repository for reproducibility. The class-weighted CNN performed best, but the logistic regression baseline remained competitive. The main value of the project is therefore not only the final score, but the structured and explainable process used to obtain it.

A PneumoniaMNIST data: example images

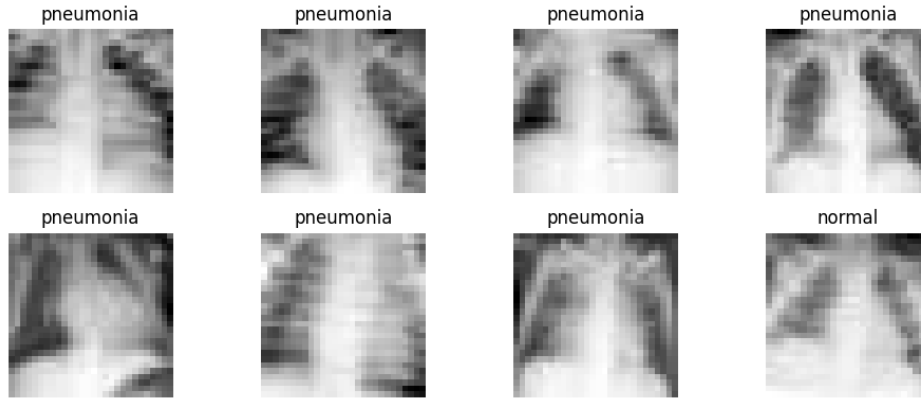


Figure 2: Example PneumoniaMNIST images. The images are low-resolution chest x-rays, which makes the task realistic for a small programming project but also limits the visual detail available to the models.

B Exact run commands

To reproduce the project locally:

```
pip install -r requirements.txt

python src/train_baseline.py
python src/train_cnn.py
python src/train_cnn_weighted.py
python src/evaluate.py
python src/plots.py
```

To run the CNN training job on Snellius using SLURM:

```
sbatch slurm/train_cnn.slurm
```

The code uses a GPU automatically when CUDA is available; otherwise it falls back to CPU. Snellius is not used automatically: it is used only when the repository is cloned on Snellius and the SLURM job is submitted.

C Detailed repository structure and file tree

configs/	configuration files for experiments
src/	model training, evaluation, metrics, plots, and utilities
notebooks/	exploratory development notebook
experiments/	final experiment summary table
outputs/	generated checkpoints, logs, and raw metrics
figures/	plots used in the report
reports/	final report and discussion
tests/	lightweight validation tests
slurm/	Snellius job scripts

```
configs/
  baseline.yaml
  cnn.yaml
  cnn_weighted.yaml
```

```
experiments/
    final_model_comparison.csv

figures/
    cnn_auc_curve.png
    cnn_loss_curve.png
    cnn_weighted_auc_curve.png
    cnn_weighted_loss_curve.png
    confusion_matrices.png
    logistic_regression_loss_curve.png
    logistic_regression_auc_curve.png

notebooks/
    pneumoniamnist_logisticregression_cnn_colab.py

outputs/
    .gitkeep

slurm/
    train_cnn.slurm

src/
    data.py
    evaluate.py
    metrics.py
    models.py
    plots.py
    train_baseline.py
    train_cnn.py
    train_cnn_weighted.py
    utils.py

tests/
    conftest.py
    test_metrics.py
    test_model_shapes.py

.gitignore
README.md
requirements.txt
```

D Training and evaluation loss/AUC figures

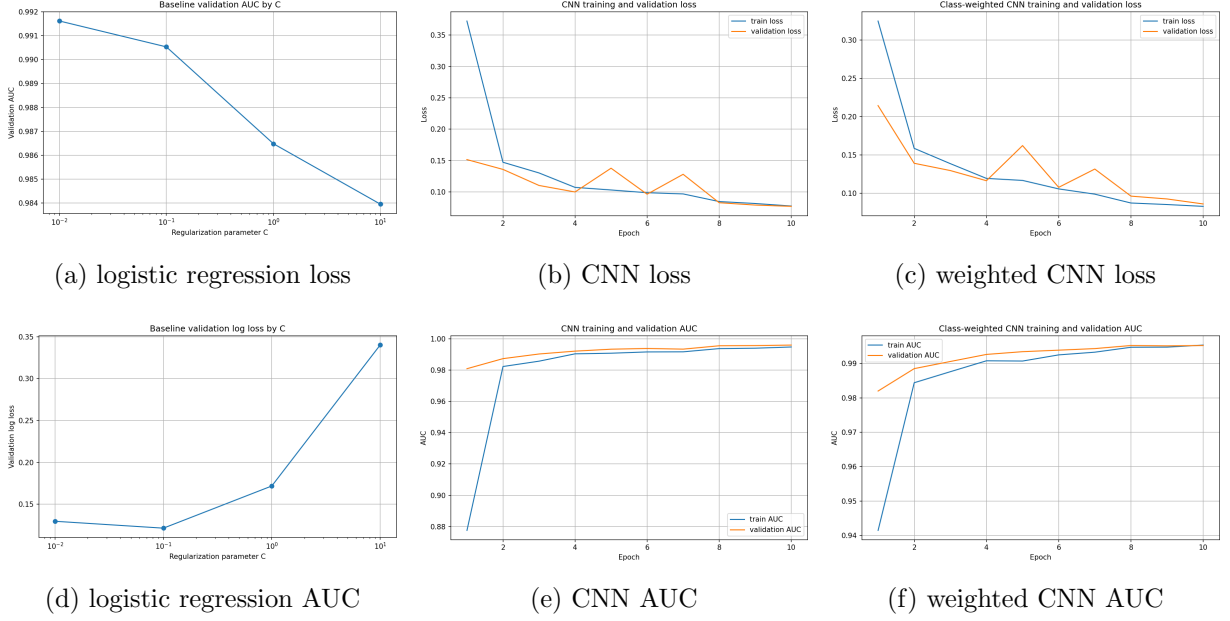


Figure 3: Training and evaluation figures of loss and AUC for logistic regression, the small CNN, and the weighted small CNN

E Class imbalance

Table 2: Class distribution across the official MedMNIST splits

Class	Training	Validation	Test
Normal	1214	135	234
Pneumonia	3494	389	390